

APUNTES SEMANALES

Josué S. Hidalgo Sandoval – 2024800128



- Actor, luchador y guardaespaldas USA (1952 –)
- Conocido como **MISTER T**
 - B. A. Baracus en The A-Team
 - Clubber Lang en Rocky III
 - Su icónica frase "I pity the fool!"
 - Su apariencia característica: mohawk y numerosas cadenas de oro

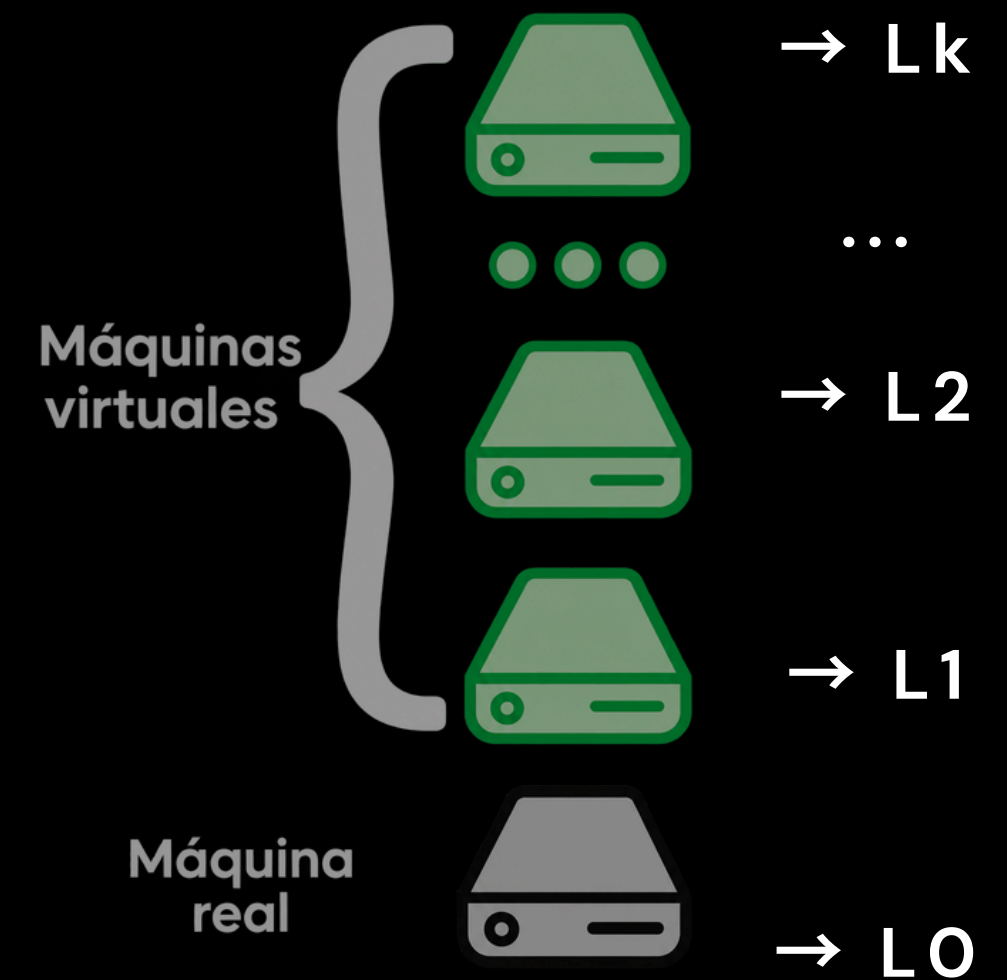
Hoy conocemos a
Mister T

The background features a dark blue gradient with several light blue line-art icons of computer monitors. One monitor is in the top left, one in the bottom right, and a stack of three monitors is in the center. The text 'MÁQUINAS Y LENGUAJES' is overlaid on the central stack of monitors.

MÁQUINAS Y LENGUAJES

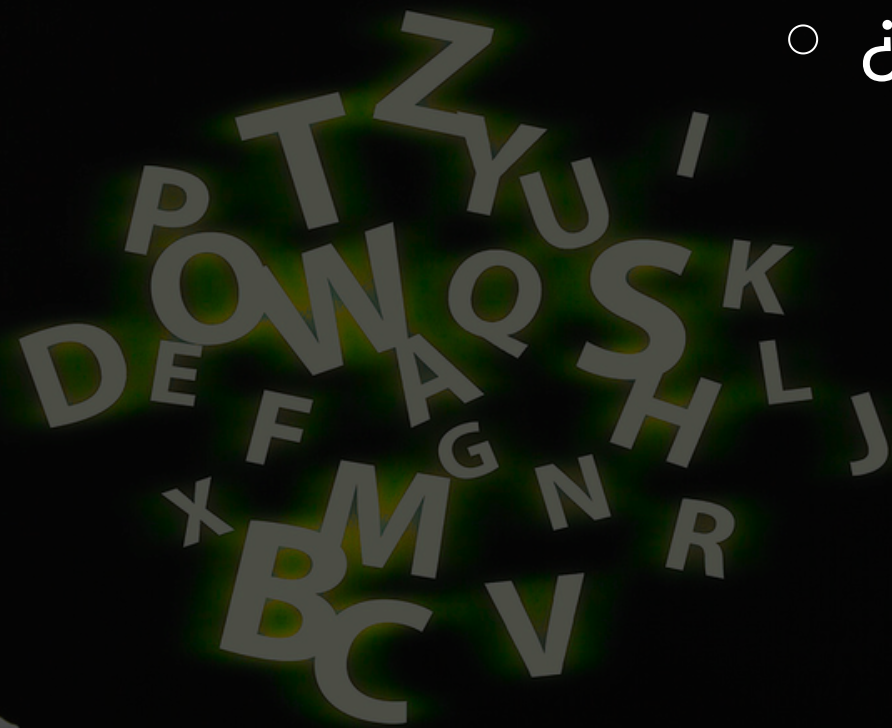
REPASO – DEFINICIONES

- **Programa:**
 - Texto formado por símbolos que describe y prescribe el comportamiento de una máquina.
- **Lenguaje de Programación:**
 - Lenguaje apropiado para escribir programas, un sistema de notación que permite describir cálculos.
 - Cada máquina "entiende" el nivel correspondiente.
- **Máquina – Lenguaje**
 - Entonces, ¿Hay una equivalencia entre
 - Máquina y Lenguaje? Pues sí... veamos



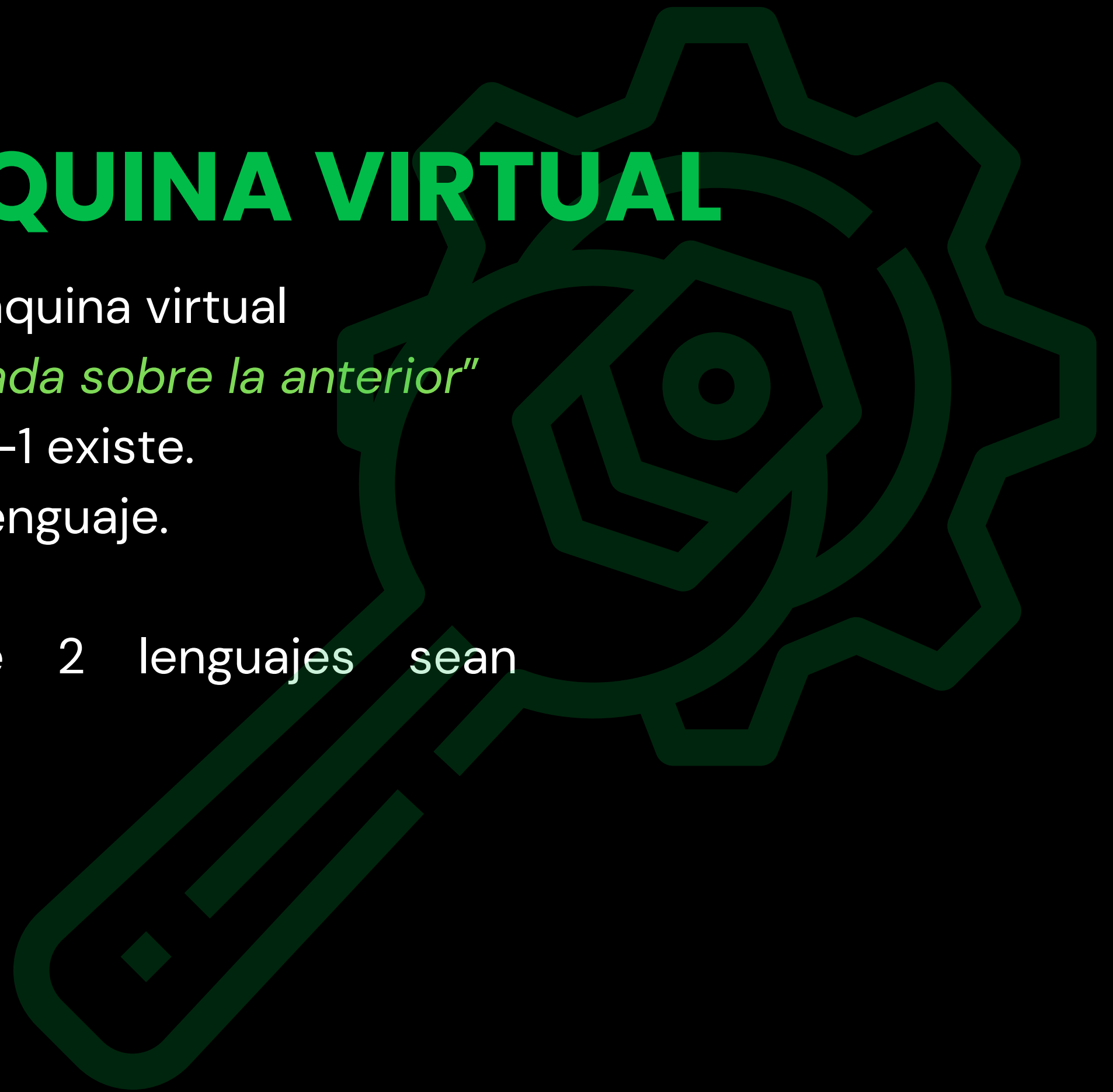
LENGUAJE

- Hay una **equivalencia** entre *Lenguaje* y *Máquina*.
- Preguntémonos:
 - ¿Qué es crear **Máquina Virtual**?
 - Diseñar el lenguaje sea entendido por otra máquina virtual o real.
 - ¿Y las **Aplicaciones**?
 - Diseñar L de la Aplicación es lograr que este Lenguaje (L-App) sea entendido por la otra máquina (que pueda ser traducido a L-K).
 - ***Que el L-App pueda ser traducido al L-K*** (Lenguaje de la siguiente máquina).



CREACIÓN DE MÁQUINA VIRTUAL

- Y dijo el creador de la siguiente máquina virtual
 - *"Cada Máquina Virtual sea creada sobre la anterior"*
- Máquina i supone que la Máquina $i-1$ existe.
- Lo que nos lleva al problema del Lenguaje.
 - $\text{Lenguaje } i \rightarrow \text{Lenguaje } i-1$
- ¿Cómo rayos hago para que 2 lenguajes sean equivalentes?
 - Los traduzco o los interpreto.
 - *Traductor (Compilador)*
 - *Intérprete*
- ¿PROBLEMA?
 - *Mapear L_i en L_{i-1}*



TRADUCCIÓN

- *Convertir completamente el programa en Lenguaje i a un programa en Lenguaje $i-1$*
- Hay que **esperar a que termine** para poder ejecutar
- **Rápida ejecución**



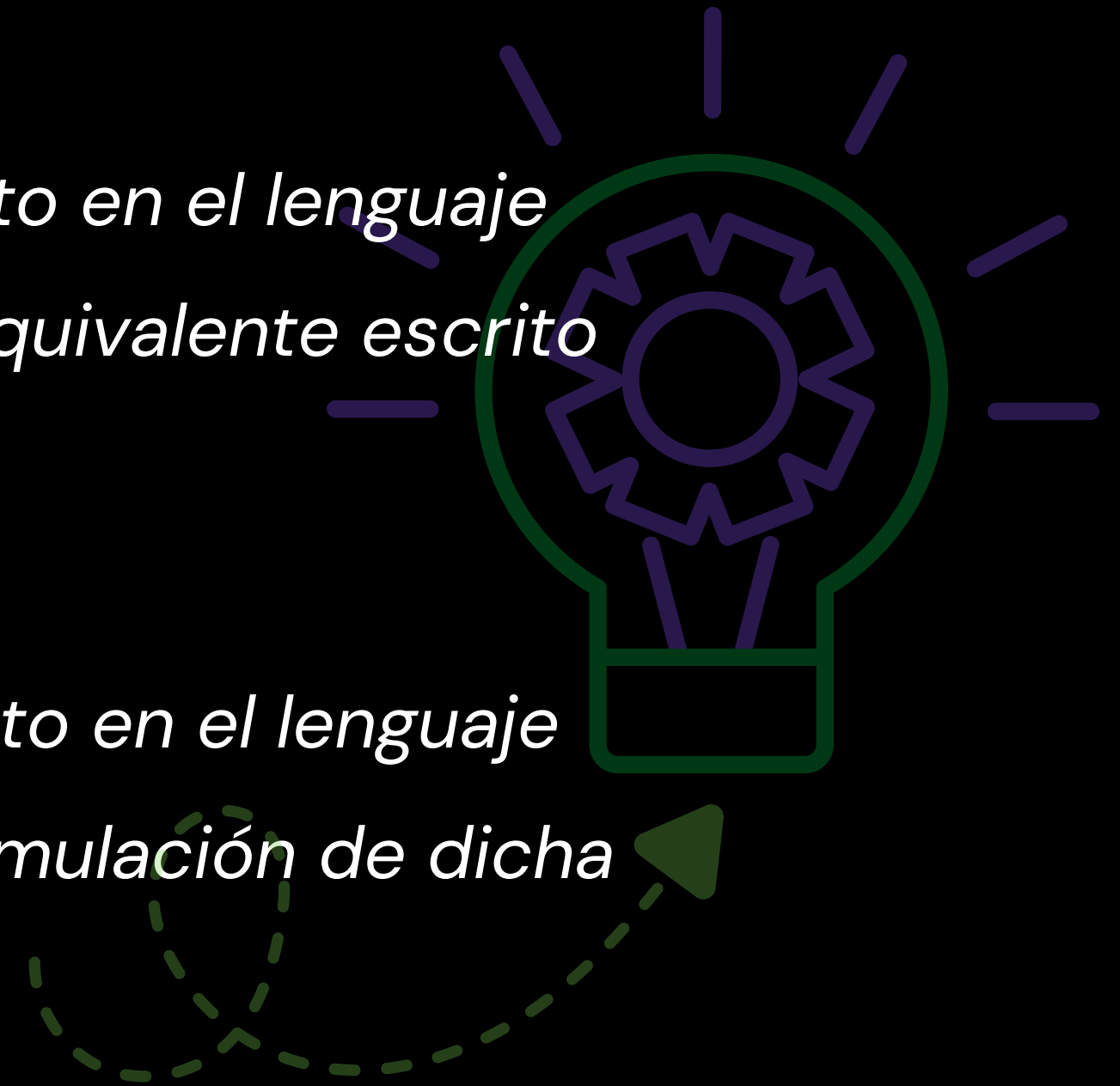


INTERPRETACIÓN

- Tomar cada instrucción que la Máquina i quiera ejecutar convertirla y ejecutarla inmediatamente.
- Simular ejecución línea por línea.

DEFINICIONES

- **Compilador:**
 - *Software que traduce un programa escrito en el lenguaje de una máquina virtual a un programa equivalente escrito en el lenguaje de otra máquina.*
- **Intérprete:**
 - *Software que ejecuta un programa escrito en el lenguaje de una Máquina Virtual corriendo una simulación de dicha máquina sobre otra máquina virtual.*





DIAGRAMAS-T

DIAGRAMA T (T-DIAGRAM)

- Hay 3 lenguajes asociados a un compilador
 - Lenguaje Fuente (**S** – Source):
 - *Entrada que debe ser traducida*
 - Lenguaje Objeto (**T** – Target):
 - *Salida hacia el que se debe traducir*
 - Lenguaje de Implementación (**H** – Host):
 - *Lenguaje en el que se está escribiendo el compilador*
- **¡¡¡Veamos EJEMPLOS!!!**



EJEMPLO 1



EJEMPLO 1



Compilador de C, que escupe x86, el cual está escrito en x86

EJEMPLO 1 – GCC



C

X86

X86

Compilador de C, que escupe x86, el cual está escrito en x86

- Antes: GNU – C – Compiler
- **Ahora: GNU – Compiler – Collection**
 - Compila varios no solo C
- Escrito originalmente por **Richar Stailman**
 - **Básicamente un terrorista**



- Registro de Activación:
 - ¿Qué putas es?
 - estructura de datos en la memoria que se crea cada vez que se llama a una función, método o procedimiento en un programa informático.
- Fuente: [Miami me lo confirmó](#)

1 Answer
Your question is very poorly stated, except for the title. There is not a single sentence that is really correct or meaningful semantically.

```
1  main(){
2      int a,b,c,d,e;
3      b = 5;
4      c = 3;
5      d = 7;
6      e = 20;
7      a = b*c + d*e;
8
9      printf("%d", a);
10 }
11
```



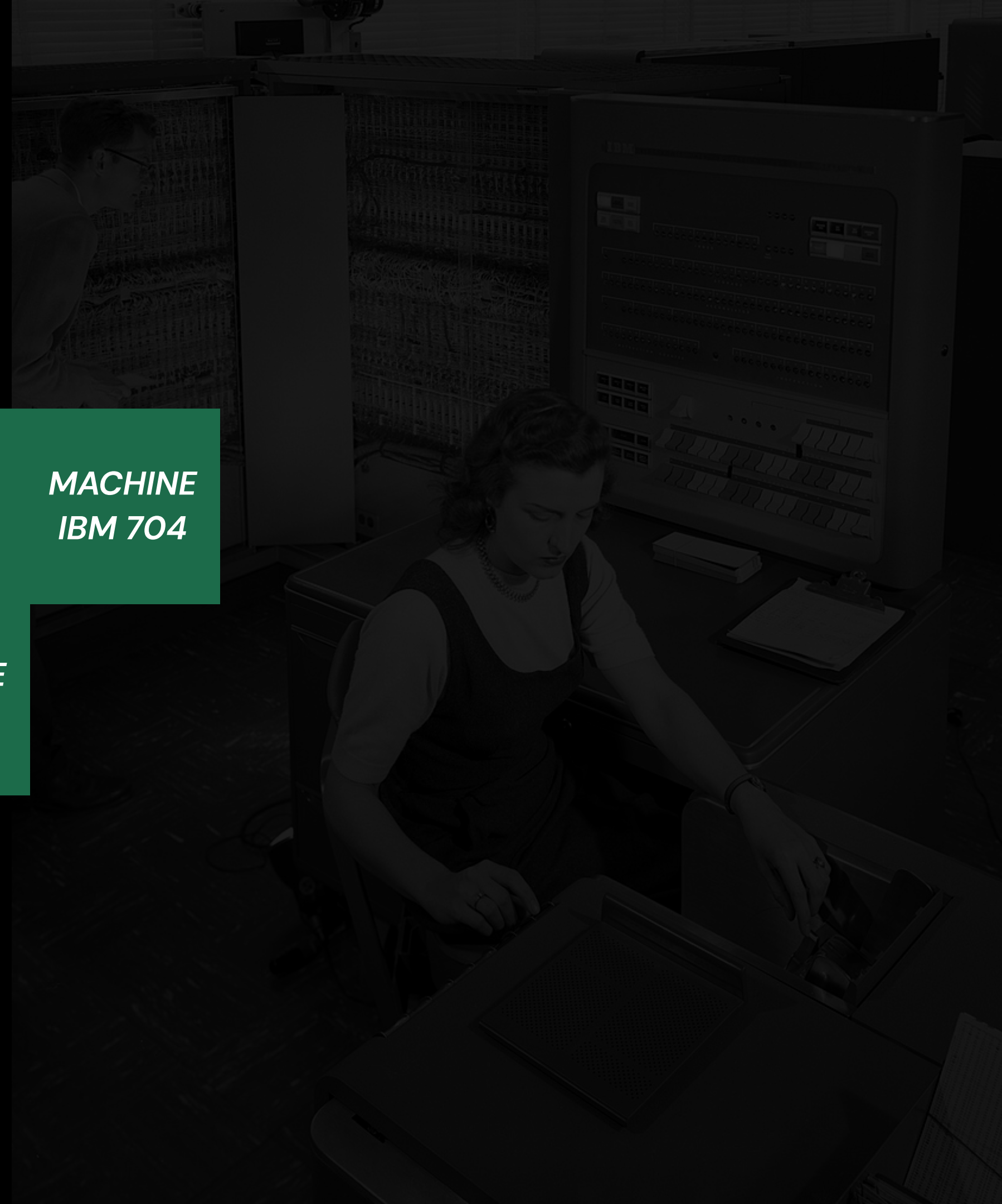
```
1  .global main
2  .extern printf
3
4  .data
5  format: .asciz "%d\n"
6
7  .text
8  main:
9      # Prologue
10     pushq   %rbp
11     movq    %rsp, %rbp
12
13     # int a, b, c, d, e;
14     # b = 5 -> %r1uc
15     movq    $5, -8(%rbp)    # b
16     # c = 3
17     movq    $3, -16(%rbp)   # c
18     # d = 7
19     movq    $7, -24(%rbp)   # d
20     # e = 20
21     movq    $20, -32(%rbp)  # e
22
23     # a = b * c + d * e;
24     movq    -8(%rbp), %rax   # rax = b
25     imulq   -16(%rbp), %rax  # rax = b * c
26
27     movq    -24(%rbp), %rcx  # rcx = d
28     imulq   -32(%rbp), %rcx  # rcx = d * e
29
30     addq    %rcx, %rax       # rax = (b * c) + (d * e)
31     movq    %rax, -40(%rbp)  # a = rax
32
33     # printf("%d", a);
34     movq    -40(%rbp), %rsi  # 2nd argument: a
35     leaq    format(%rip), %rdi # 1st argument: format string
36     movl    $0, %eax
37     call    printf
38
39     # Epilogue
40     movl    $0, %eax
41     popq    %rbp
42     ret
```

EJEMPLO 2

ASSEMBLY
IBM 704

MACHINE
IBM 704

MACHINE
IBM 704



EJEMPLO 2

ASSEMBLY
IBM 704

MACHINE
IBM 704

MACHINE
IBM 704

*Compilador de Assembly IBM 704, que escupe el Máquina de IBM 704, el cual está escrito en el Máquina de IBM 704
(Ocurrió históricamente)*

EJEMPLO 2 – SAP

ASSEMBLY
IBM 704

MACHINE
IBM 704

MACHINE
IBM 704

- Symbolic Assembly Program
- Es un programa ensamblador para el ordenador IBM 704
- Escrito por Ray Nutt
- Distribuido por el grupo de usuarios de SHARE
- Influenció a los ensambladores del futuro

MONITOR FOR 6802 1.4 9-14-80 TSC ASSEMBLER PAGE 2

```
C000          ORG      ROM+$0000 BEGIN MONITOR
C000 8E 00 70  START  LDS      #STACK

                                *****
                                * FUNCTION: INITA - Initialize ACIA
                                * INPUT: none
                                * OUTPUT: none
                                * CALLS: none
                                * DESTROYS: acc A

0013          RESETA EQU      %00010011
0011          CTLREG EQU      %00010001

C003 86 13      INITA  LDA A  #RESETA  RESET ACIA
C005 B7 80 04          STA A  ACIA
C008 86 11          LDA A  #CTLREG  SET 8 BITS AND 2 STOP
C00A 86 04          STA A  ACIA
C00D 7E C0 F1          JMP      SIGNON  GO TO START OF MONITOR

                                *****
                                * FUNCTION: INCH - Input character
                                * INPUT: none
                                * OUTPUT: char in acc A
                                * DESTROYS: acc A
                                * CALLS: none
                                * DESCRIPTION: Gets 1 character from terminal

C010 B6 80 04  INCH   LDA A  ACIA      GET STATUS
C013 47          ASR A                  SHIFT RDRF FLAG INTO CARRY
C014 24 F0      BCC  INCH  IF NOT CARRY
C016 86 04      LDA A  ACIA      GET STATUS
C019 84 7F      AND A  #$7F      MASK PARITY
C01B 7E C0 79      JMP      OUTCH  ECHO & RTS

                                *****
                                * FUNCTION: INHEX - INPUT HEX DIGIT
                                * INPUT: none
                                * OUTPUT: Digit in acc A
                                * CALLS: INCH
                                * DESTROYS: acc A
                                * Returns to monitor if not HEX input

C01E 8D F0      INHEX  BSR      INCH      GET A CHAR
C020 81 30          CMP A  #'0          ZERO
C022 2B 11          BMI      HEXERR      NOT HEX
C024 81 39          CMP A  #'9          NINE
C026 2F 0A          BLE      HEXRTS      GOOD HEX
C028 81 41          CMP A  #'A
C02A 2B 09          BMI      HEXERR      NOT HEX
C02C 81 46          CMP A  #'F
C02E 2E 05          BGT      HEXERR
C030 80 07          SUB A  #7          FIX A-F
C032 84 0F      HEXRTS AND A  #$0F      CONVERT ASCII TO DIGIT
C034 39          RTS

C035 7E C0 AF  HEXERR JMP      CTRL      RETURN TO CONTROL LOOP
```

EJEMPLO 3



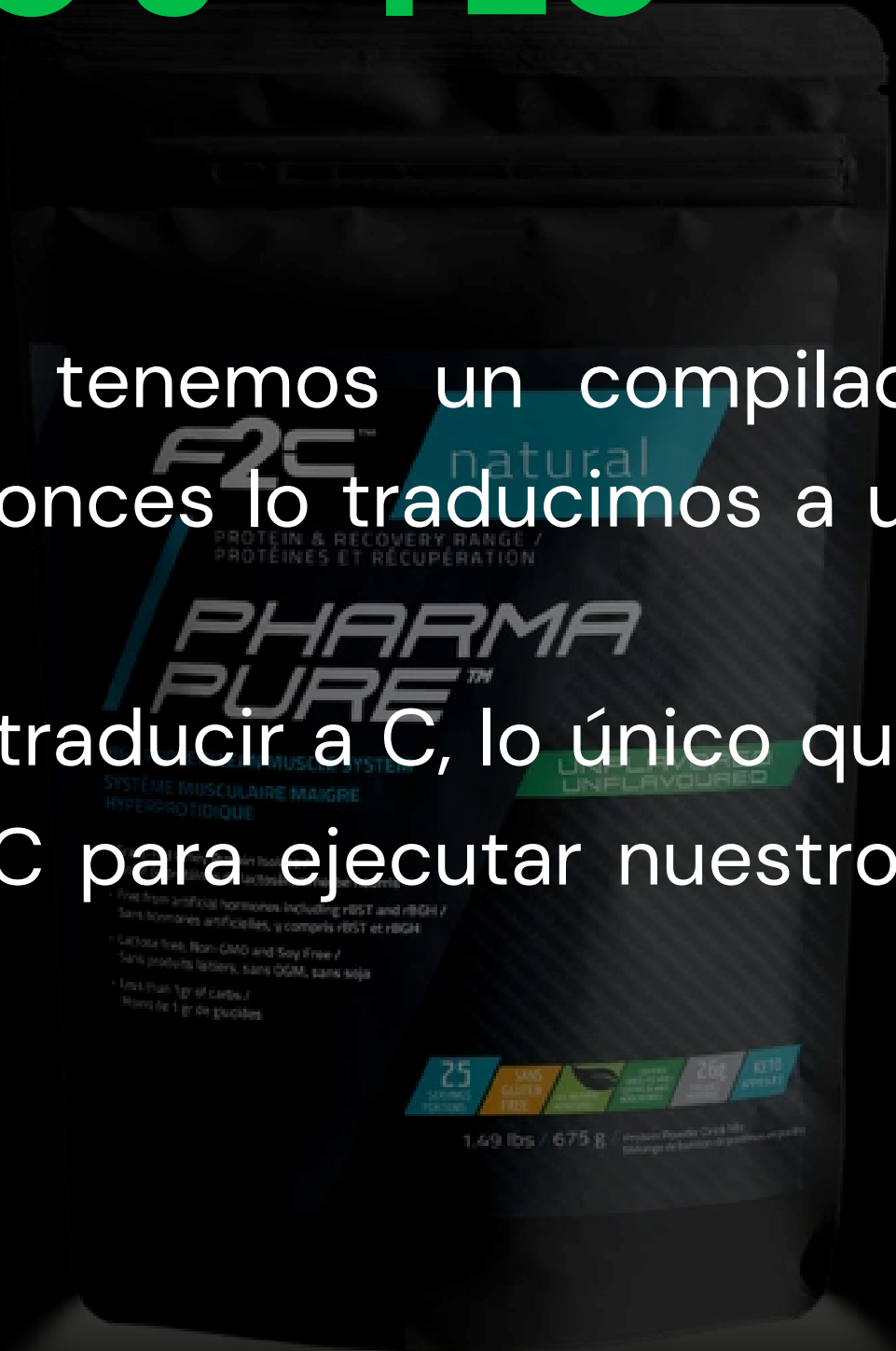
EJEMPLO 3



Compilador de FORTRAN, que escupe el C, el cual está escrito en x86

EJEMPLO 3 – F2C

- A veces no tenemos un compilador de cierto lenguaje entonces lo traducimos a uno del cual sí tenemos.
- Entonces al traducir a C, lo único que hacemos es compilar el C para ejecutar nuestro programa en FORTRAN.



FORTRAN

C

X86



ÁLGEBRA DIAGRAMAS T



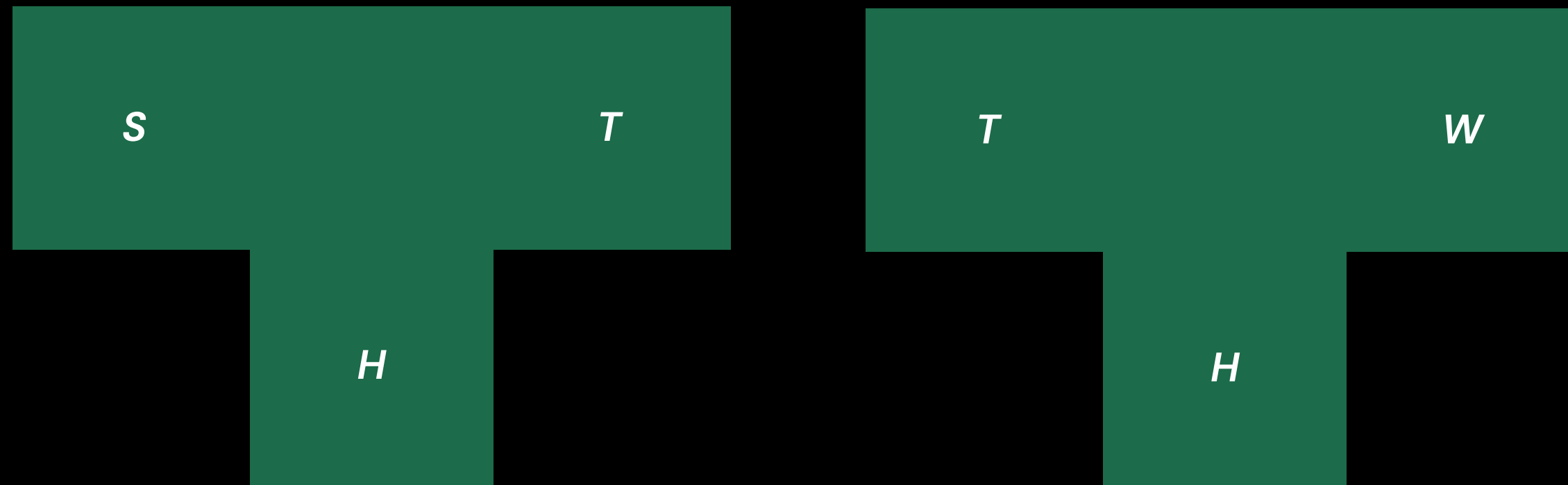


ÁLGEBRA DIAGRAMAS T

Podemos **combinar**
los **Diagramas T** para
representar **más**
complejos

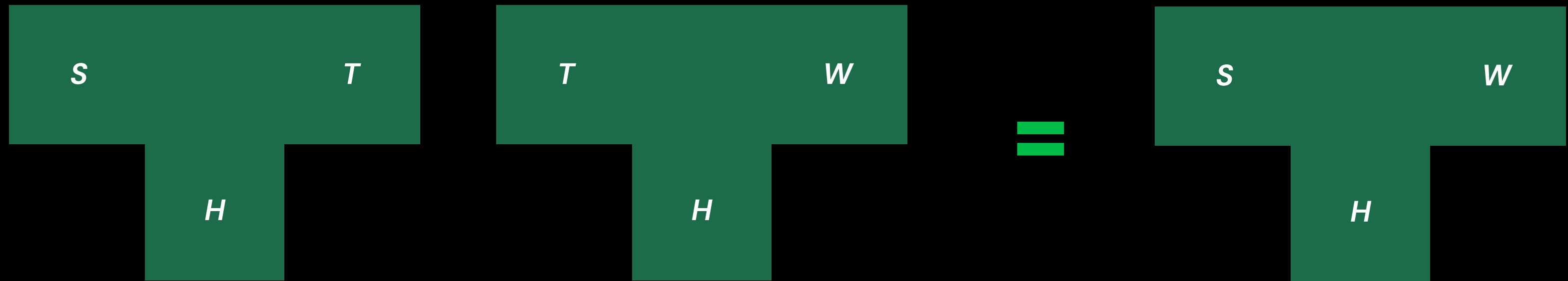
PRIMER CASO - ($T == S$)

- Lenguaje Objeto (T) es igual a Lenguaje Fuente (S) y entonces se pueden **pegar**



PRIMER CASO - ($T == S$)

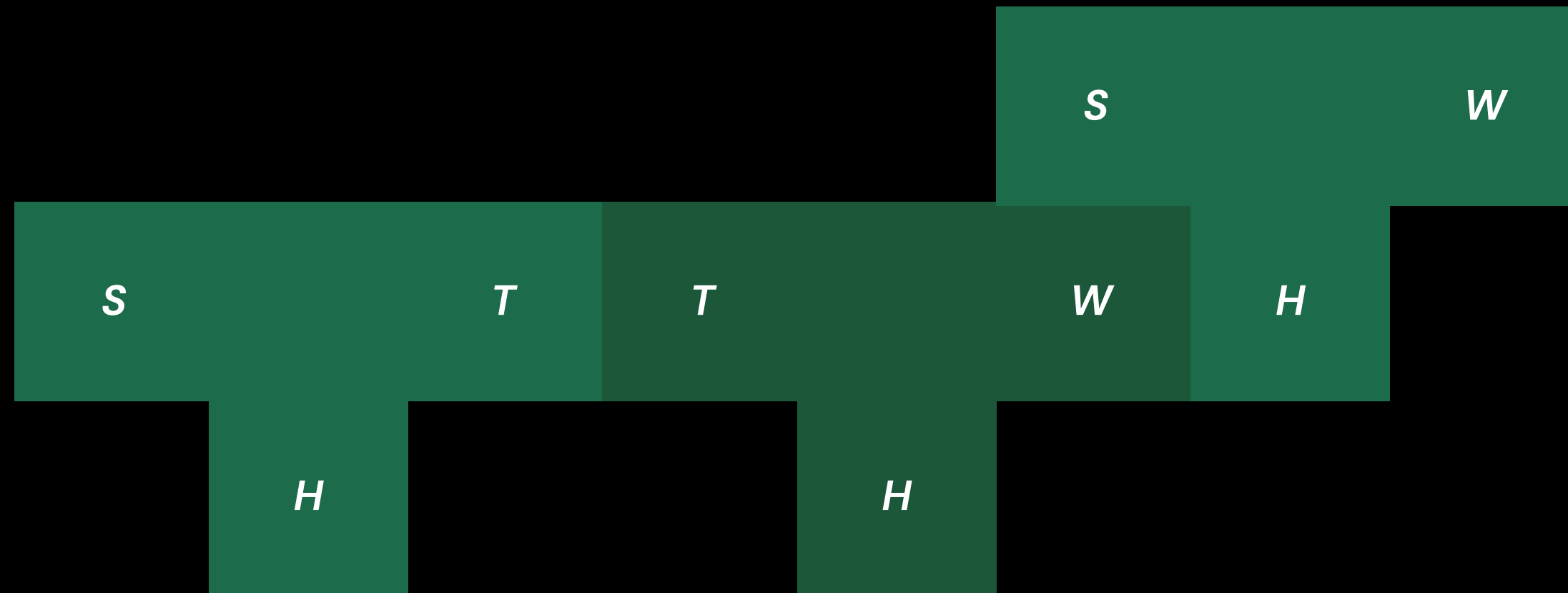
- Lenguaje Objeto (T) **del primero** es igual a Lenguaje Fuente (T) **del segundo** y entonces se pueden **pegar**



- No me convence esa notación, hagámos una notación OFICIAL

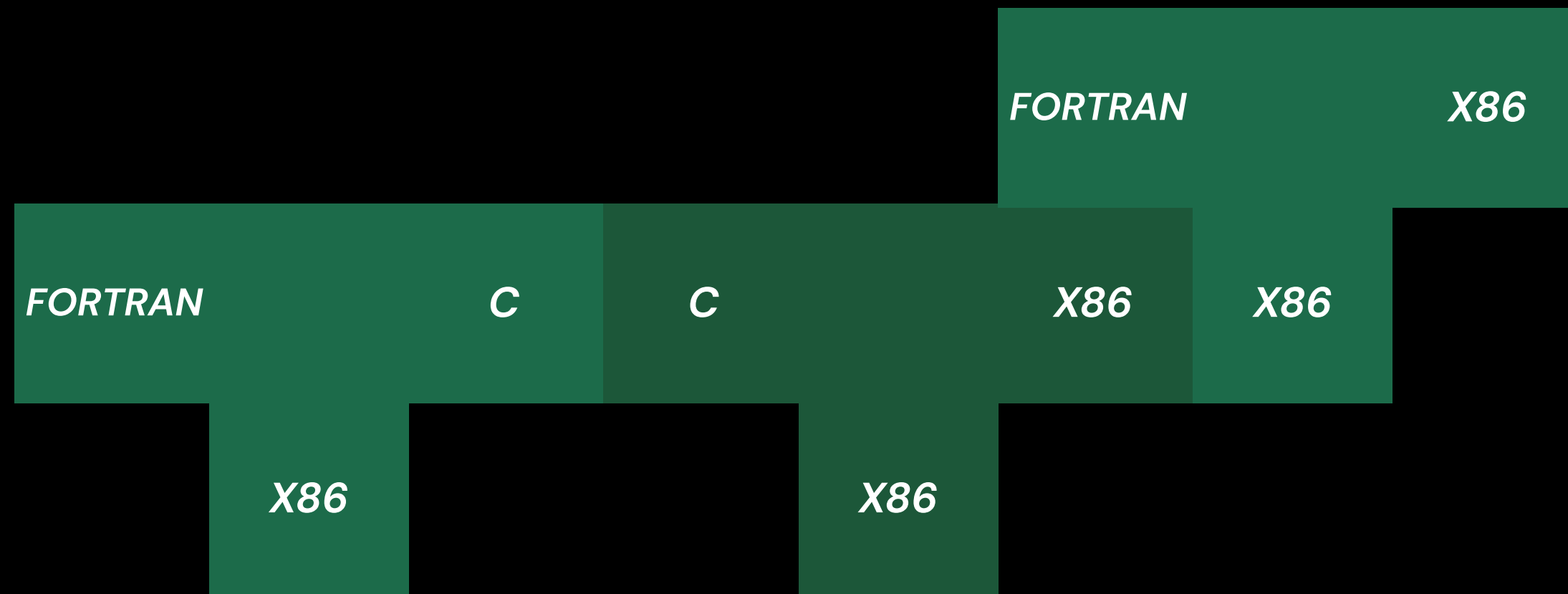
PRIMER CASO – (T == S)

- NOTACIÓN
 - para los Compiladores Virtuales o Compiladores Pipeados (Paipados)



PRIMER CASO – (T == S) – EJEMPLO 1

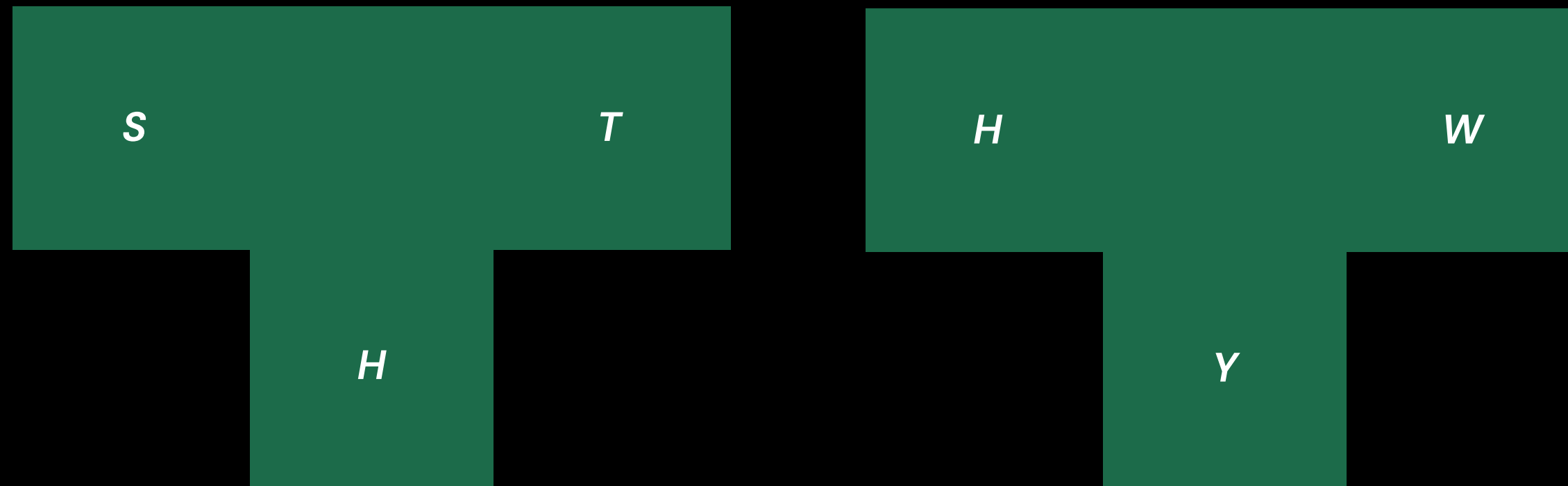
- F2C y GCC
 - Nos da un compilador virtual de Fortran



- Escribir un compilador es difícil
- ¿En qué **Lenguaje de Implementación** lo escribimos MICRO? XD, NO el apropiado

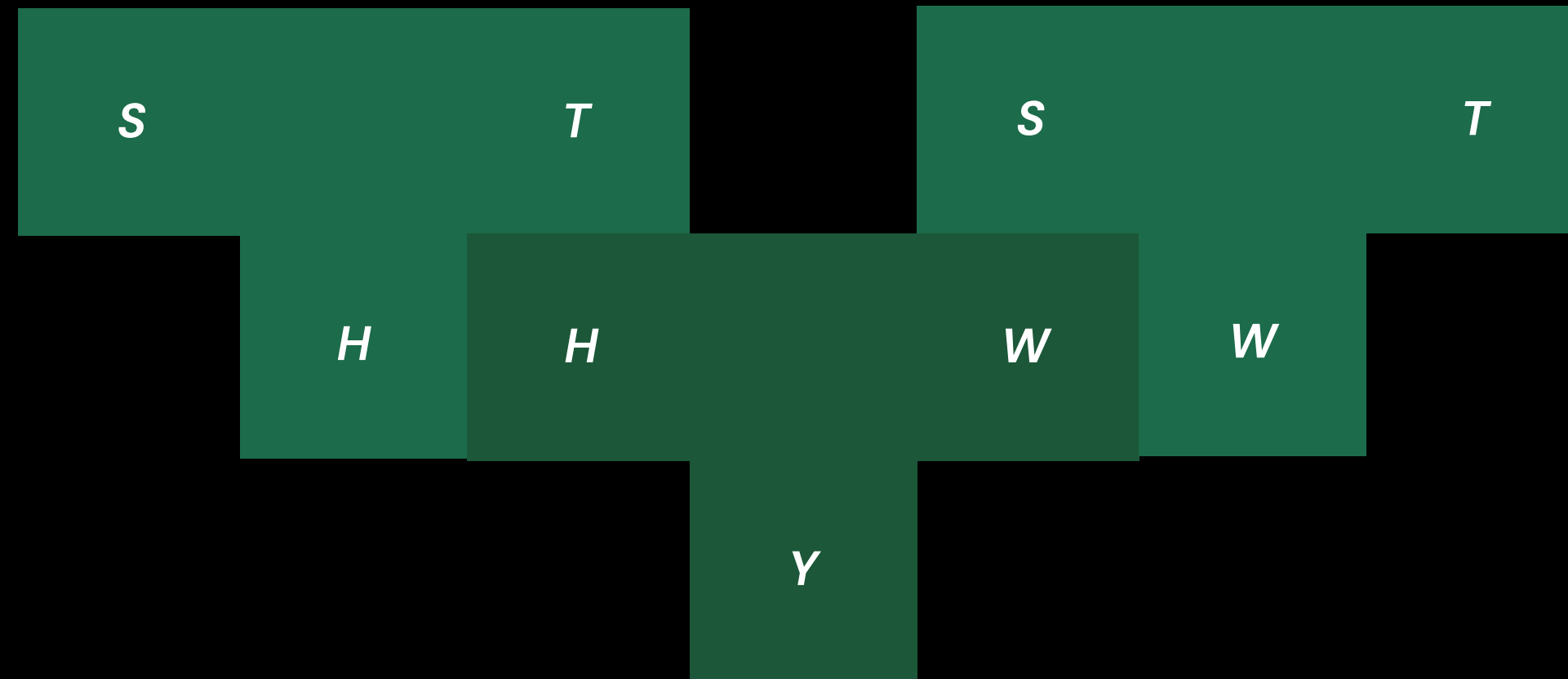
SEGUNDO CASO – ($H == S$)

- Lenguaje de Implementación (H) del **primero** es igual a Lenguaje Fuente (H) del **segundo** y entonces se pueden **encadenar**



SEGUNDO CASO – ($H == S$)

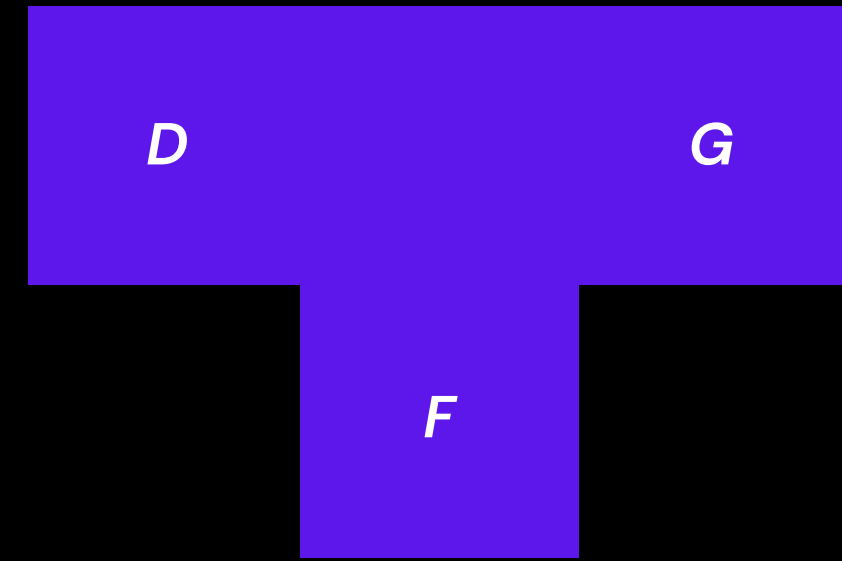
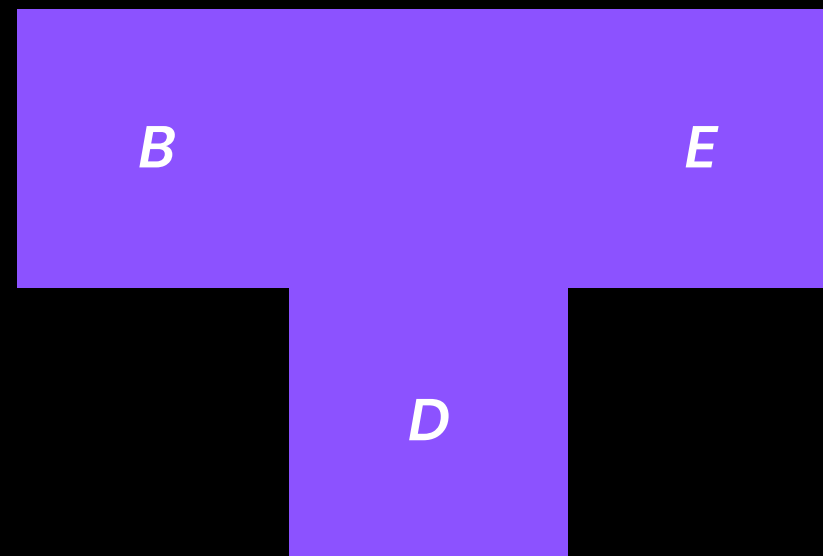
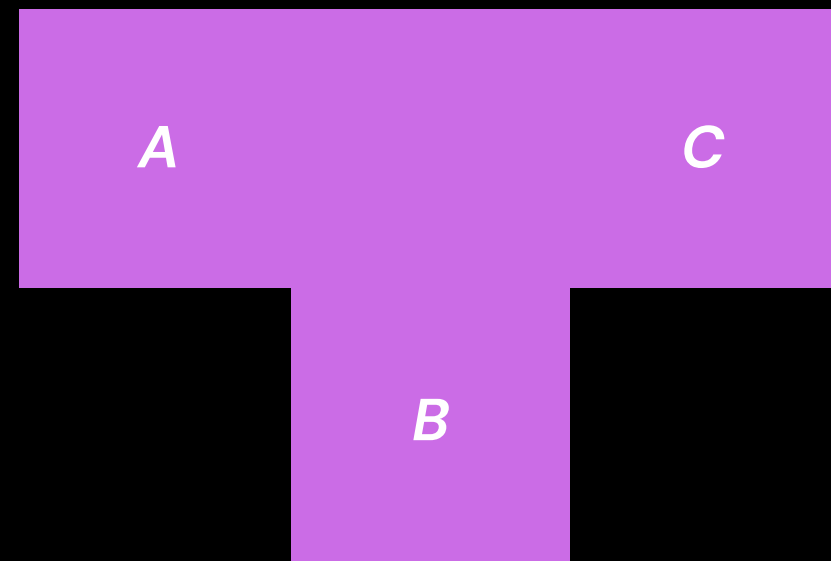
- NOTACIÓN



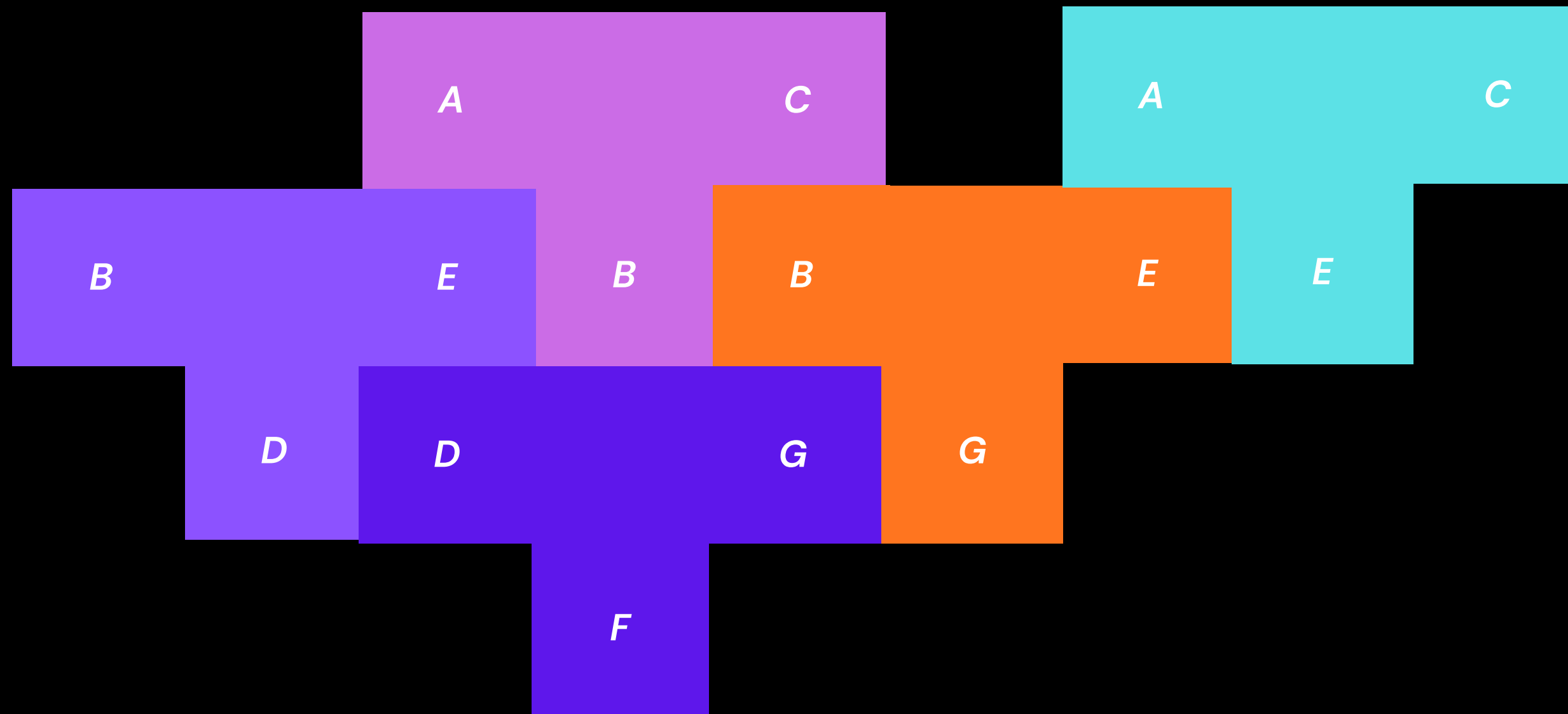
Bonito :)

¿Y SI TENEMOS 3 COMPILADORES?

- Hagámos algo con estos 3 chunches

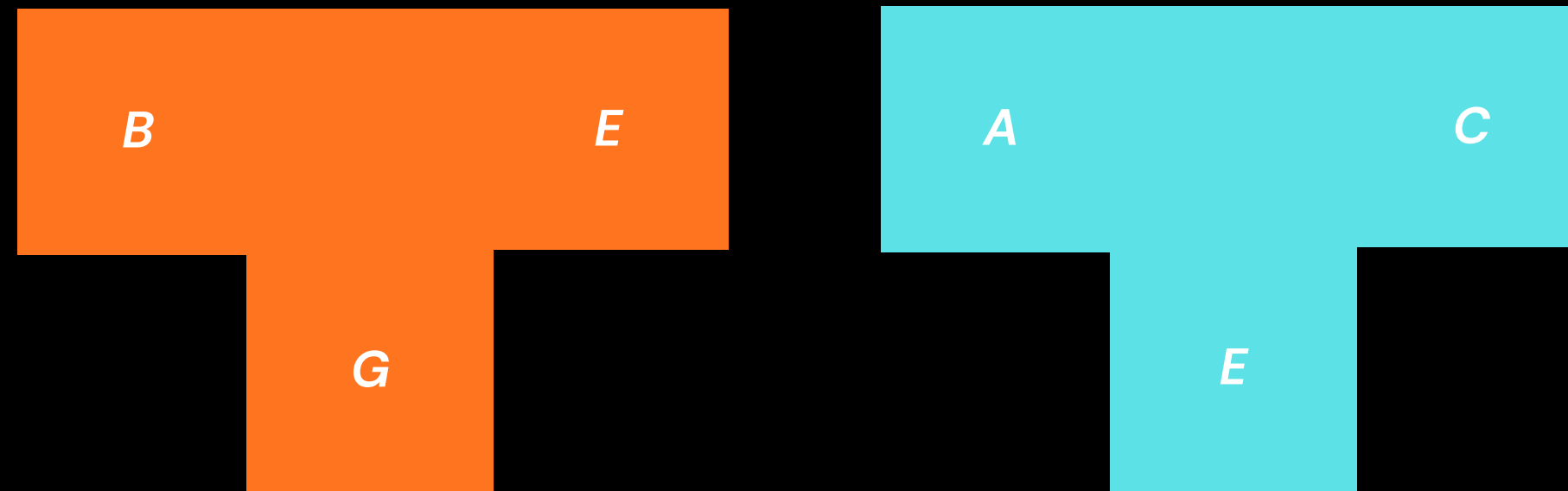


¿Y SI TENEMOS 3 COMPILADORES?



¿Y SI TENEMOS 3 COMPILADORES?

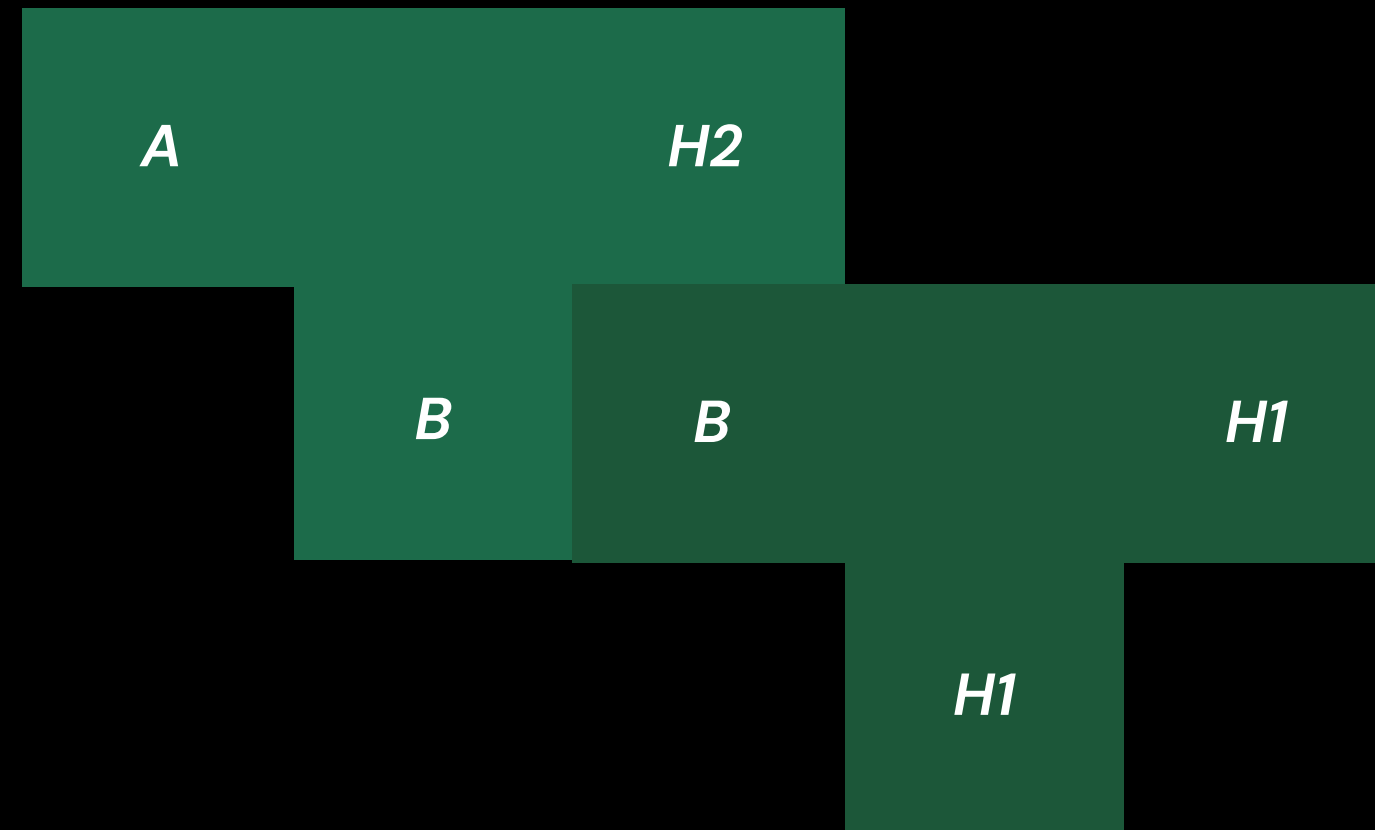
- Obtuve 2 chunces más



- El **Lenguaje Objeto**: normalmente se genera código máquina, pero qué sino...

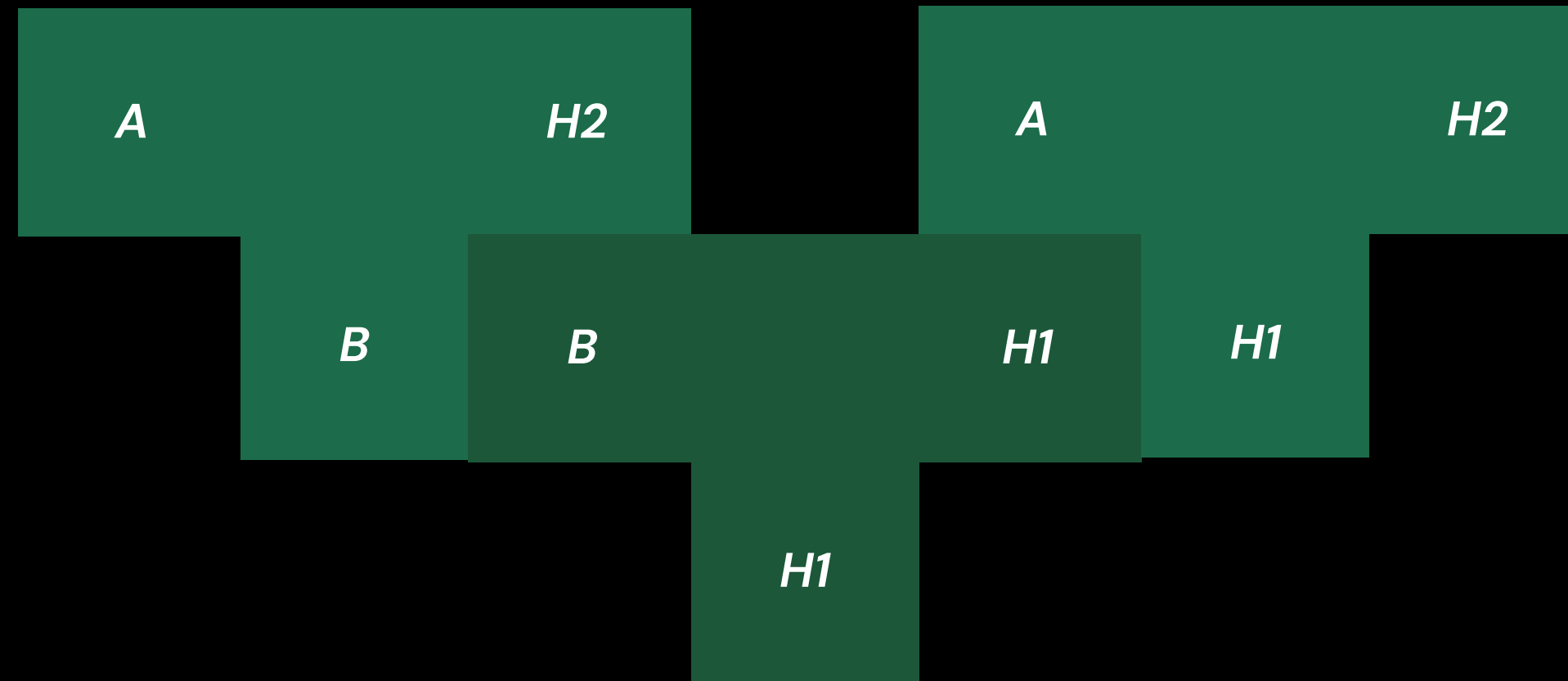
TERCER CASO – (T $\neq$ H)

- Lenguaje Objeto (T) puede ser diferente a la máquina en la que corre el compilador (H)



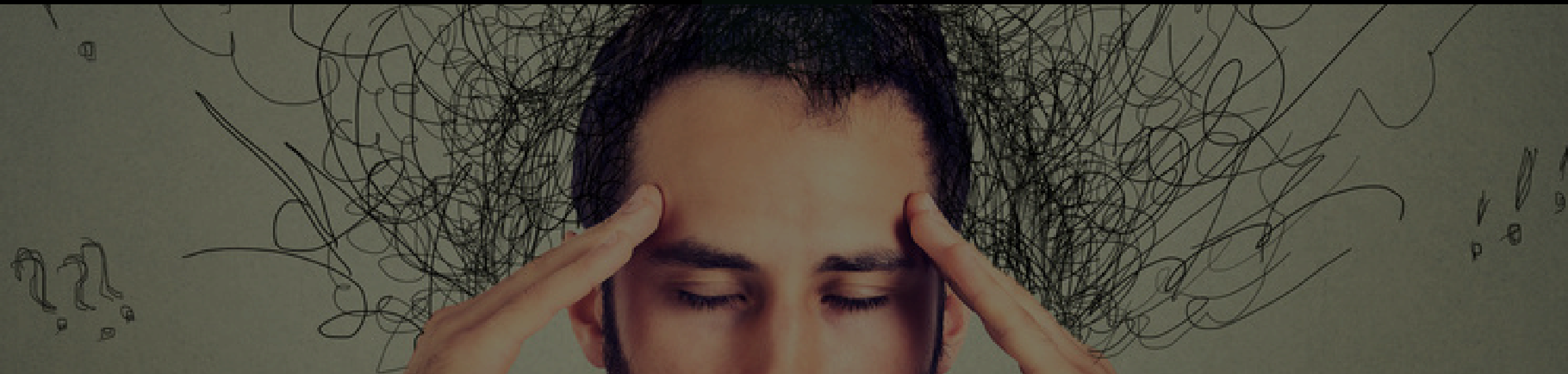
TERCER CASO – (T != H)

- Lenguaje Objeto (T) puede ser diferente a la máquina en la que corre el compilador (H)

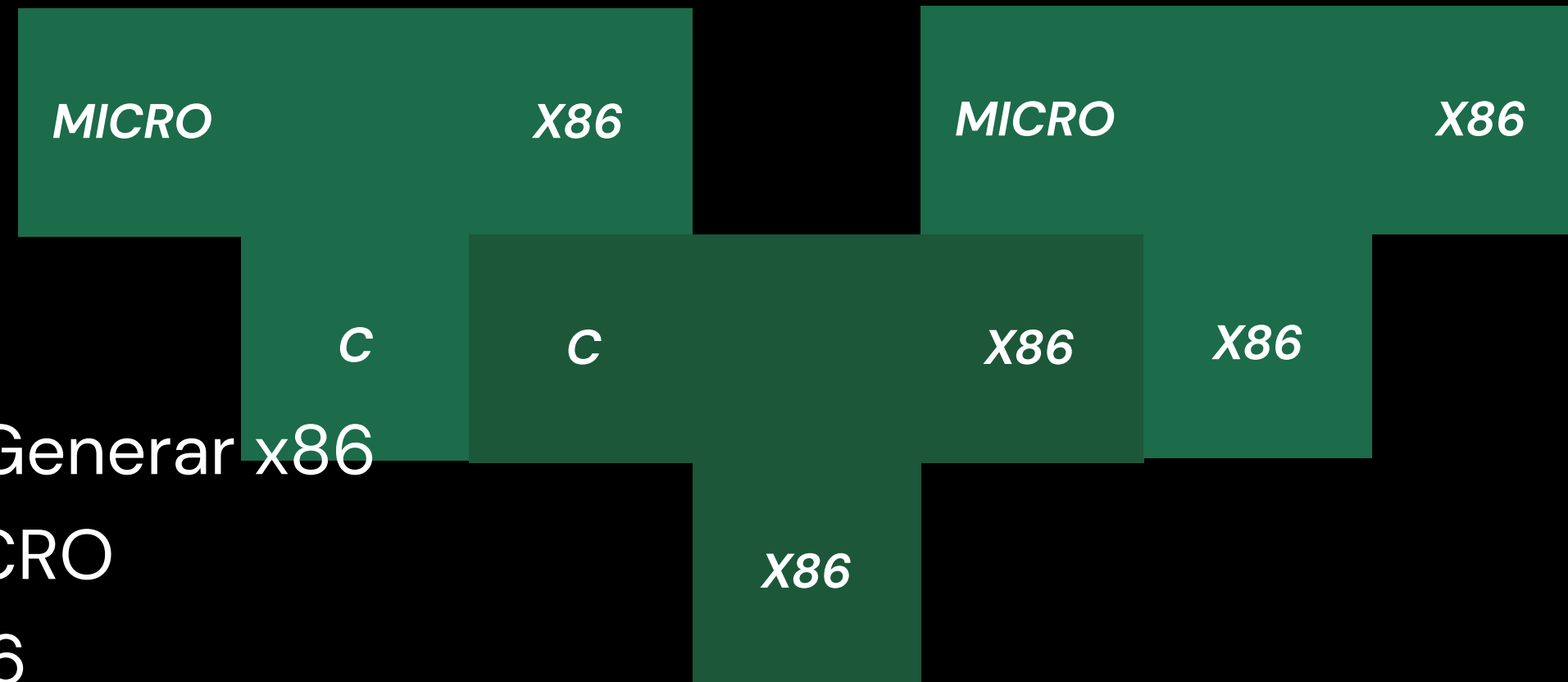


PROYECTO 0

- Recibe código MICRO
- Escrito en C
- Según el semestre:
 - **VARIANTE A:** Generar x86
 - **VARIANTE B:** Generar MIPS (con simulador)
 - **VARIANTE C:** Generar ARM (con simulador)

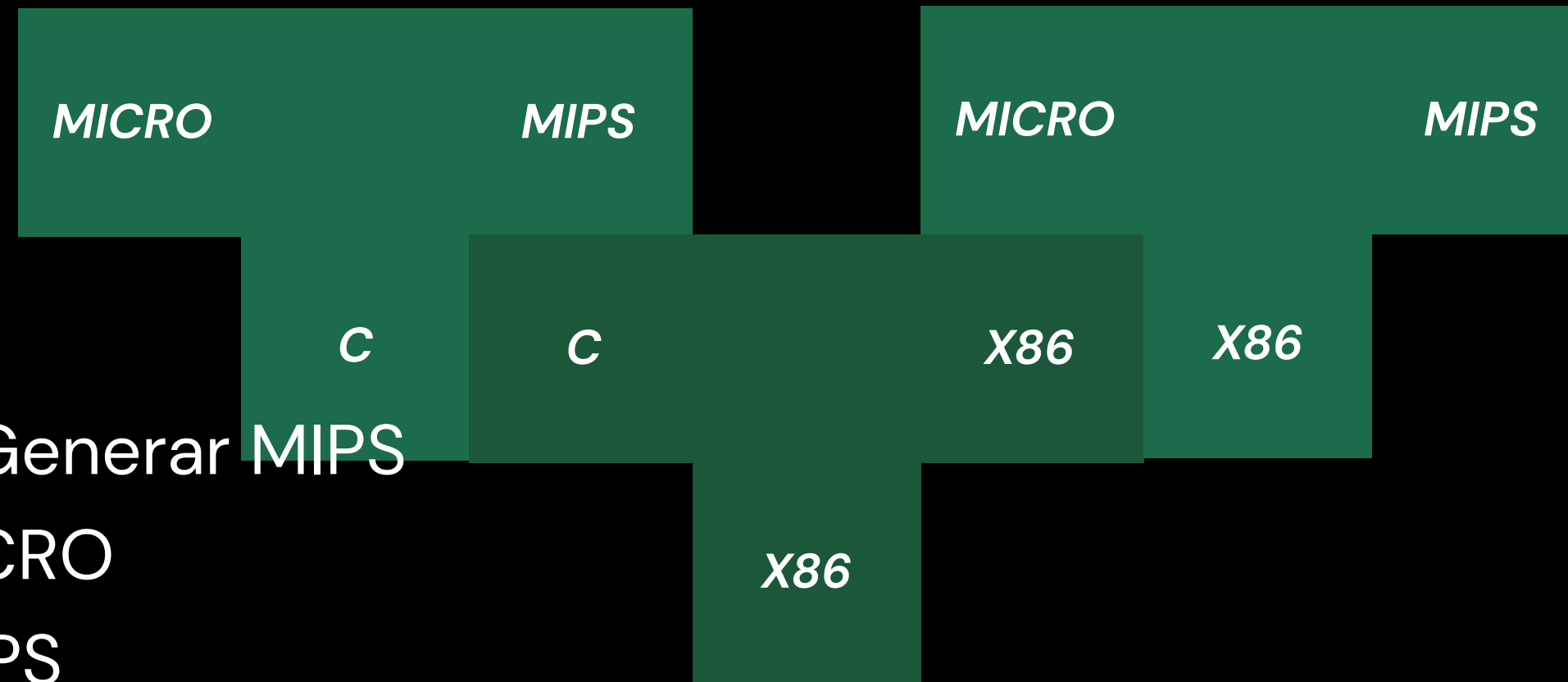


PROYECTO 0



- **VARIANTE A:** Generar x86
 - Recibe MICRO
 - Genera x86
 - Escrito en C
 - Compilado sobre GCC

PROYECTO 0

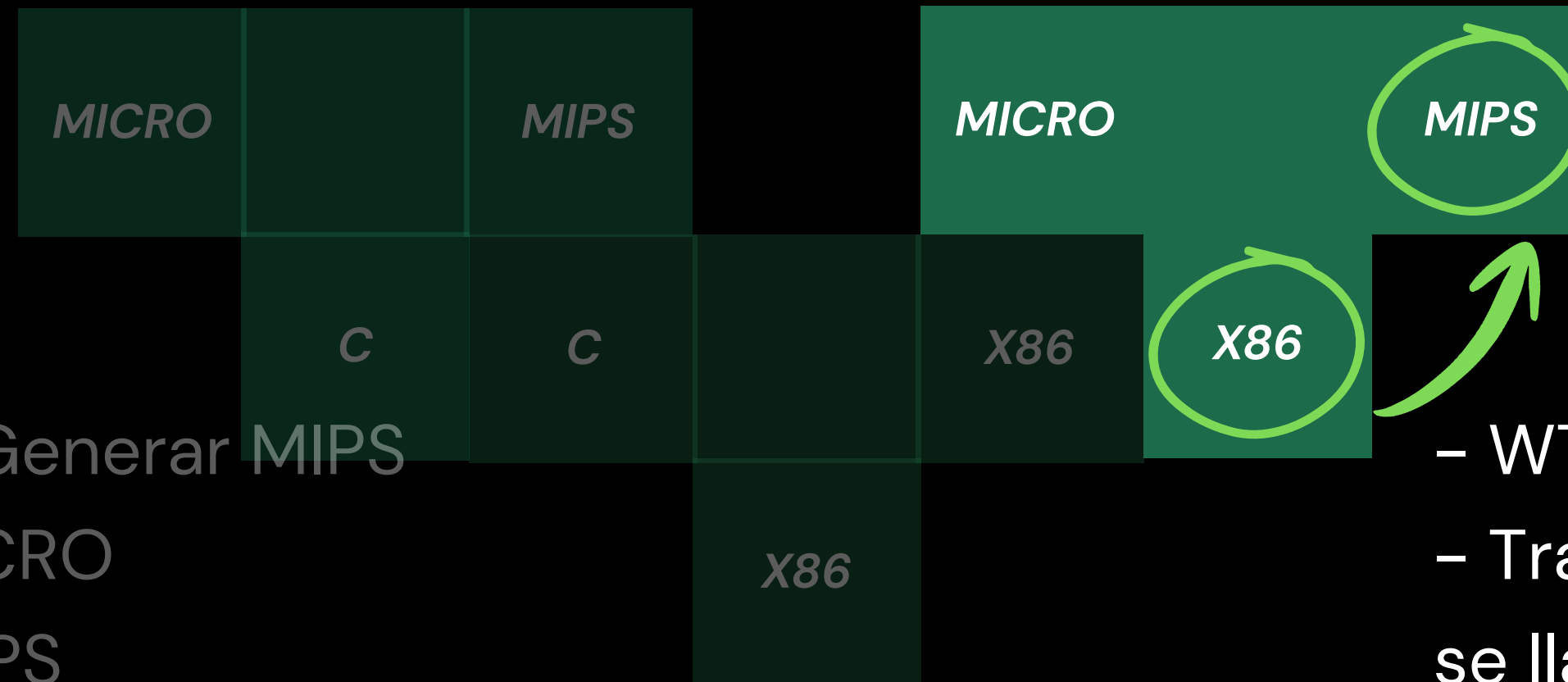


- **VARIANTE B:** Generar MIPS
 - Recibe MICRO
 - Genera MIPS
 - Escrito en C
 - Compilado sobre GCC

ARQUITECTURA MIPS

- Arquitectura RISC
 - Diseño original de **MIPS Company Systems** (1985)
 - Versiones I, II, III, IV, V
 - Arquitectura **influyente para diseño RISC**
 - Estudiada en algún Curso
 - ¿Simulador conocido de MIPS? **SPIM** (anagrama)
 - Compilamos con GCC
-
- **VARIANTE B:** Generar MIPS
 - Recibe MICRO
 - Genera **MIPS**
 - Escrito en C
 - Compilado sobre GCC

PROYECTO 0



- **VARIANTE B:** Generar MIPS

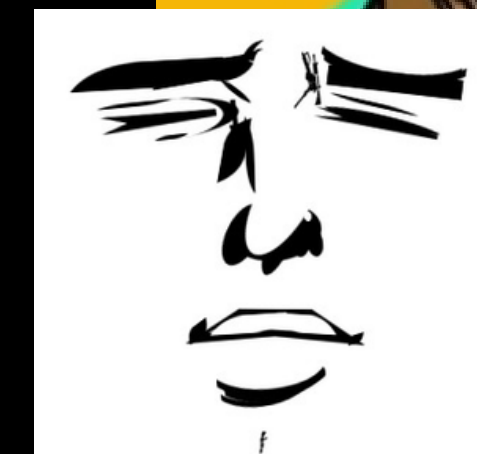
- Recibe MICRO
- Genera MIPS
- Escrito en C
- Compilado sobre GCC

– WTF?

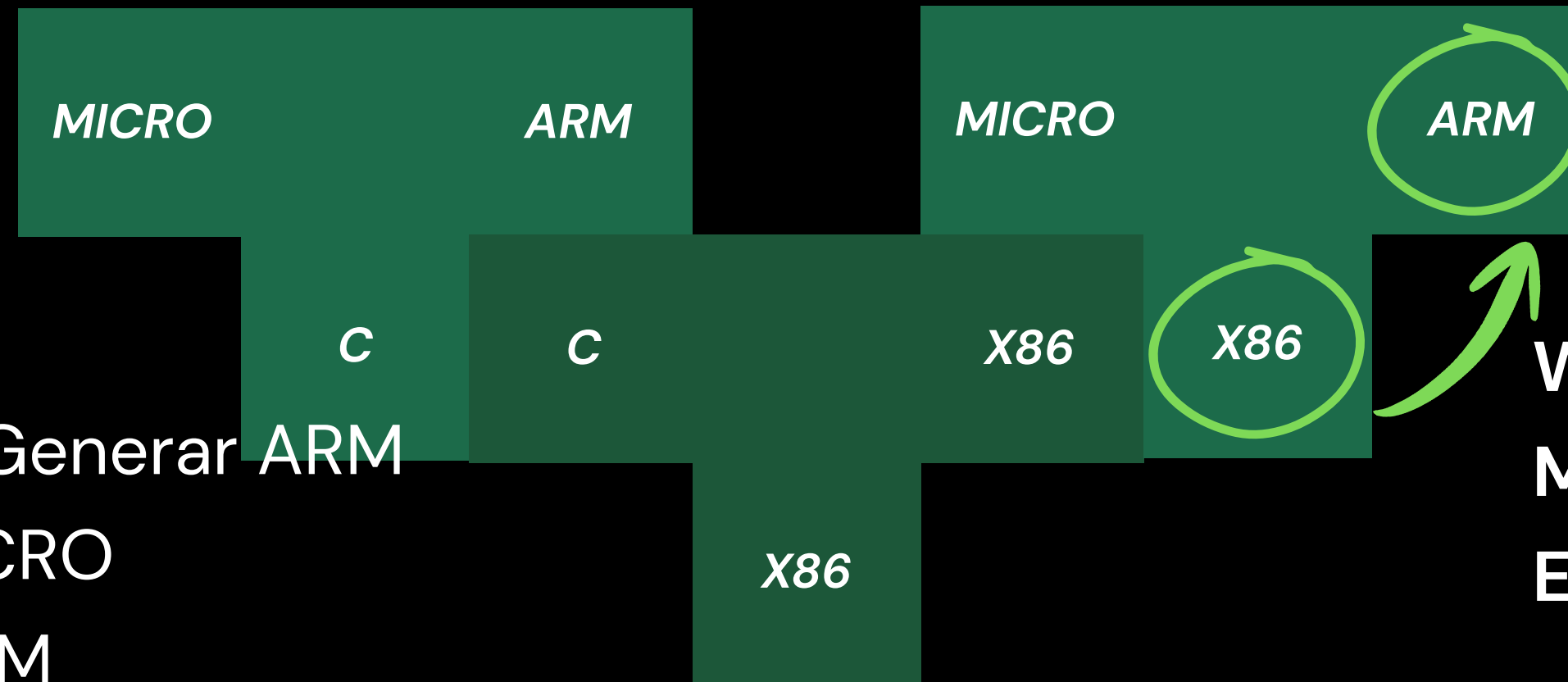
– Tranqui ya explico esto
se llama: **CROS COMPILER**

CROSS COMPILER

- *Situación donde el código Objeto por un compilador no es el mismo que el lenguaje máquina donde corre.*
- Máquina 1 host $\neq$ Máquina 2 (Target)
- **¿Cuándo se necesita?**
 - No tengo ésta máquina
 - Aún ésta máquina no existe
 - Máquina destino tiene pocos recursos
 - Sistemas **empotrados**
 - Escrito en C
 - Compilado sobre GCC



PROYECTO 0



- **VARIANTE C:** Generar ARM

- Recibe MICRO
- Genera ARM
- Escrito en C
- Compilado sobre GCC

ARQUITECTURA ARM

- Familia de Arquitecturas
- **Arquitectura RISC**
- **Diseño original 1980 – Británica**
- **Muy eficiente en espacio y energía**
- Muy usadas en celulares, laptops, tablets.
- Sistemas Empotrados
- ARM holdings los diseña, pero no las fabrica

- **VARIANTE C:** Generar ARM

- Recibe MICRO

- Genera **ARM**

- Escrito en C

- Compilado sobre GCC

- Modelos con 32 o 64 bits

- Es la **Arquitectura más usada**

- 325 mill millones al 2026

- 99% de los smartphones

- 30% a 40% de todos los dispositivos electrónicos

APUNTES SEMANALES

Josué S. Hidalgo Sandoval – 2024800128



- Actor, luchador y guardaespaldas USA (1952 –)
- Conocido como **MISTER T**
 - B. A. Baracus en The A-Team
 - Clubber Lang en Rocky III
 - Su icónica frase "I pity the fool!"
 - Su apariencia característica: mohawk y numerosas cadenas de oro

Hoy conocemos a
Mister T